Proyecto final – Rutas del Paisaje Cultural Cafetero

1. Introducción

En este proyecto cada grupo construirá un sistema de planificación de rutas sobre la red vial real del Quindío y analizará, de forma teórica y experimental, los algoritmos que lo componen.

El Quindío hace parte del Paisaje Cultural Cafetero, declarado Patrimonio Mundial por la UNESCO en 2011. Sus 12 municipios están unidos por una red de vías que conecta cabeceras municipales, veredas, fincas cafeteras y atractivos turísticos. Sobre esa red aparecen problemas algorítmicos concretos: encontrar la ruta más rápida entre dos puntos, aproximar la extensión de la red, decidir en qué orden recorrer varios lugares y comprobar si un plan de visitas cabe en un día.

Cada problema corresponde a un tema del curso. Algunos se resuelven de forma eficiente con algoritmos clásicos; otros, como el problema del viajante, solo se resuelven de forma exacta con algoritmos de costo exponencial, y eso obliga a elegir entre exactitud y tiempo de cómputo. La idea es que usted mida esas diferencias con datos reales y no solo con ejemplos de tablero.

2. Propósito del proyecto

Implementar y analizar algoritmos de grafos, geometría computacional, programación dinámica, algoritmos voraces, divide y vencerás y heurísticas sobre la red vial del Quindío, y contrastar su complejidad teórica con mediciones reales.

Al terminar el proyecto, el grupo debe ser capaz de:

1. Modelar un problema real como un grafo y elegir la estructura de datos adecuada.
2. Diseñar experimentos reproducibles y comprobar si lo medido coincide con lo que predice la teoría.
3. Comparar estrategias (fuerza bruta, voraz, divide y vencerás, programación dinámica y heurísticas) sobre un mismo problema, en calidad de la solución y en tiempo de ejecución.

3. Condiciones generales

- Lenguaje: libre (por ejemplo Python, C++, Rust, Go, Elixir o Java), pero todos los algoritmos deben estar en el mismo lenguaje para que las mediciones sean comparables. La descarga de la red se hace en Python, porque OSMnx es una librería de Python; si los algoritmos se implementan en otro lenguaje, el grafo y el catálogo se exportan a archivos (GraphML, CSV o JSON) y se leen desde allí.
- Implementación propia: Cada requerimiento especifica qué algoritmos deben implementarse desde cero, sin usar funciones de librería que los resuelvan. Se permite usar librerías para las estructuras de apoyo y las operaciones de ordenamiento, cuyo costo debe investigarse e incluirse en el análisis. Para el resto del proyecto, como la preparación de datos, las gráficas y la interfaz, se pueden utilizar librerías.
- Librerías: se pueden usar librerías para descargar y leer datos (OSMnx, GeoPandas), para graficar (Matplotlib, Plotly, Folium, Leaflet) y para la interfaz. En los algoritmos de implementación propia, librerías como NetworkX, SciPy o Shapely solo pueden usarse para comprobar resultados, salvo que el requerimiento indique otra cosa.
- Fuentes: cuando el enunciado pide una fuente (por ejemplo, para la complejidad de un algoritmo o para los datos del catálogo), se cita en el mismo texto, con autor y título o con el enlace.
- Inteligencia artificial: está permitida como apoyo, pero debe declararse en un anexo del documento técnico: qué herramientas se usaron, para qué y cómo se verificaron sus respuestas. Cada integrante debe poder explicar y modificar cualquier parte del código en la sustentación.
- Desarrollo guiado por especificaciones: Se promueve el uso de Spec-Driven Development (SDD), definiendo antes de programar cada requerimiento una especificación breve con su comportamiento esperado y casos de prueba. Esta se guarda en el repositorio (por ejemplo, en `specs/`), sirve como guía para la IA y permite verificar que el código cumpla lo solicitado.
- Repositorio: el código fuente del proyecto debe estar en un repositorio de GitHub compartido con el docente. El historial de commits se usa como evidencia del trabajo de cada integrante.

4. Datos

El proyecto usa dos fuentes de datos: la red vial de OpenStreetMap y un catálogo de lugares que arma cada grupo.

Red vial. Se descarga con OSMnx, por ejemplo con `ox.graph_from_place("Quindío, Colombia", network_type="drive")`. El resultado es un grafo dirigido: los nodos son puntos de la red, principalmente intersecciones y extremos de vías (con latitud y longitud), y las aristas son tramos de vía (con su longitud en metros, el tipo de vía y, a veces, la velocidad máxima). El grupo debe indicar la fecha de descarga. Si la descarga falla, el docente puede entregar un archivo GraphML.

Catálogo de lugares. Cada grupo arma un catálogo de al menos 40 lugares del departamento:

Categoría	Mínimo	Ejemplos
Cabeceras municipales	12	Armenia, Buenavista, Calarcá, Circasia, Córdoba, Filandia, Génova, La Tebaida, Montenegro, Pijao, Quimbaya, Salento
Atractivos turísticos	15	Parque Nacional del Café, PANACA, Valle de Cocora, Jardín Botánico del Quindío, Museo del Oro Quimbaya
Fincas cafeteras u hoteles rurales	8	A elección del grupo
Centros de acopio o trilladoras de café	5	A elección del grupo

De cada lugar se registra: identificador, nombre, municipio, categoría, latitud y longitud. De cada atractivo turístico se registra además la duración estimada de la visita (en minutos), que se usa en el requerimiento 4 y se justifica con una fuente o con un criterio definido por el grupo. Las coordenadas se verifican en un mapa. Las cabeceras municipales se ubican en su plaza principal; la de Armenia (plaza de Bolívar) es el punto de salida y llegada por defecto del requerimiento 4.

5. Requerimientos

Para cada algoritmo se pide implementarlo, verificar que es correcto, indicar su complejidad en tiempo y en memoria y medirlo según el protocolo de la sección 6. La complejidad de cada algoritmo se consulta en una fuente y se cita en el documento técnico; en los algoritmos recursivos, de divide y vencerás o de programación dinámica (como Held-Karp), se acompaña de su recurrencia.

Requerimiento 1. Modelo de la red vial

1. Representación. Descargar la red vial del Quindío. Con el número real de nodos (V) y aristas (E), estimar cuántos megabytes ocuparía la red como matriz de adyacencia y como listas de adyacencia, indicando los supuestos del cálculo (por ejemplo, cuántos bytes ocupa cada celda o cada entrada). Con esos números, elegir una de las dos representaciones y justificar la elección.
2. Pesos. Guardar en cada arista su distancia (m) y su tiempo de viaje (s), calculado a partir de la distancia y la velocidad del tramo. Explicar cómo se obtuvo la velocidad de cada tramo y cómo se trataron los tramos sin velocidad. El tiempo así calculado es una estimación del modelo, no el tiempo real de recorrido.

3. Componente fuertemente conexa. Una componente fuertemente conexa es un conjunto de nodos en el que, respetando el sentido de las vías, se puede ir de cualquiera a cualquier otro. Obtener la componente fuertemente conexa más grande con `ox.truncate.largest_component(G, strongly=True)`. Investigar qué algoritmo usa internamente, consultando la documentación o el código fuente de la versión utilizada. Explicar brevemente cómo funciona e indicar su complejidad en tiempo y en memoria. Medir cuánto tarda la función en redes de tamaño creciente: las redes alrededor de la plaza de Bolívar de Armenia descargadas con `ox.graph_from_point` con radios de 2, 5, 10 y 20 km, y la red completa del departamento. Presentar en una tabla el tamaño de cada red (V + E) y el tiempo medido, y comprobar si coinciden con la complejidad investigada. El resto del proyecto trabaja solo con esa componente; explicar por qué eso conviene para los requerimientos siguientes.

4. Ubicación de los lugares. Asociar cada lugar del catálogo al nodo de la componente que esté más cerca en línea recta (distancia euclidiana entre sus coordenadas en metros, UTM zona 18N). Ese nodo será el origen o destino de las rutas hacia el lugar. Reportar los lugares cuyo nodo quede a más de 500 m y explicar cómo se trataron.

5. Grafo procesado. Guardar la componente, sus pesos y las asociaciones de los lugares para reutilizarlos sin descargar nuevamente la red. Reportar el número de nodos y aristas antes y después de filtrar la componente.

Requerimiento 2. Caminos mínimos

Se trabaja sobre la componente del requerimiento 1, usando como peso el tiempo de viaje.

1. Dijkstra. Implementar Dijkstra (implementación propia) en dos versiones: buscando el mínimo con un recorrido lineal, $O(V^2)$, y con una cola de prioridad, $O((V + E) \log V)$. Ambas se detienen al sacar el nodo destino y devuelven, además del tiempo, la ruta (la secuencia de nodos), reconstruida a partir del predecesor de cada nodo. Esa ruta es la que se dibuja en la aplicación del requerimiento 5. Cada nodo se cuenta como explorado una sola vez, la primera vez que sale de la cola.
   - Enunciar el invariante que hace correcto a Dijkstra, explicar por qué permite detenerse al sacar el destino y qué condición deben cumplir los pesos para que se mantenga.
   - Medir la versión con cola de prioridad con al menos 20 pares origen-destino del catálogo, y la versión lineal con 10 de esos pares, elegidos entre lugares cercanos (del mismo municipio o de municipios vecinos), porque en la red completa puede tardar varios minutos por consulta. Registrar el tiempo y los nodos explorados en cada consulta.
   - Presentar una gráfica de tiempo frente a nodos explorados, con las dos versiones en la misma figura. En los 10 pares medidos con ambas, calcular el cociente entre el tiempo de la versión lineal y el de la versión con cola de prioridad, y reportar su mediana, mínimo y máximo.

- Matriz de tiempos. Calcular y guardar la matriz de tiempos de viaje entre todos los lugares del catálogo, incluida Armenia (al menos 40 × 40). La matriz no es necesariamente simétrica, pues el tiempo de ir de un lugar a otro puede diferir del tiempo de regreso. Para calcularla se usa la versión con cola de prioridad, ejecutada una vez desde cada lugar sin detenerse en un destino. Esta matriz se usa en el requerimiento 4.

- Verificación. Comprobar que las dos versiones de Dijkstra den los mismos tiempos en todos los pares medidos con ambas.

Requerimiento 3. Envolvente convexa de la red vial

La envolvente convexa es el menor polígono convexo que contiene un conjunto de puntos. Calculada sobre los nodos de la componente, vistos como puntos en el plano, da una aproximación de la extensión de la red vial.

Coordenadas. Convertir las coordenadas a metros (UTM zona 18N), aproximar cada coordenada al entero más cercano y eliminar los puntos que queden repetidos. Esto permite calcular giros y alineaciones usando coordenadas enteras. En lenguajes con enteros de tamaño fijo, utilizar enteros de 64 bits para evitar desbordamientos. Los puntos alineados sobre los lados del polígono no se cuentan como vértices.

1. Algoritmo. Elegir e implementar (implementación propia) un algoritmo de envolvente convexa de divide y vencerás, justificando la elección según la distribución de los puntos de la red. Calcular también la envolvente con `scipy.spatial.ConvexHull` y verificar que ambos den los mismos vértices en todas las pruebas.
2. Experimento. Medir las dos versiones con los nodos de la red y con puntos al azar dentro de un cuadrado (1 000, 10 000 y 50 000 puntos, con semilla fija). Presentar los tiempos en una tabla, compararlos y explicar la diferencia, investigando qué algoritmo y qué lenguaje usa SciPy internamente.

Requerimiento 4. Ruta óptima de recorrido

Se sale de un lugar del catálogo, se visitan k lugares más y se regresa al lugar de partida, buscando el menor tiempo total según la matriz del requerimiento 2. Es el problema del viajante en su versión asimétrica, porque ir de i a j puede tardar distinto que ir de j a i. En los experimentos (puntos 1 y 3), el lugar de partida es Armenia y los k lugares se eligen al azar del catálogo (con semilla fija), entre todas sus categorías.

1. Soluciones exactas. Implementar (implementación propia) la fuerza bruta (todas las permutaciones) y Held-Karp (programación dinámica sobre subconjuntos de lugares). Para cada una, aumentar k desde 5 hasta 12, o hasta que una ejecución pase el límite de tiempo de la sección 6 si eso ocurre antes, y reportar el mayor k resuelto. Presentar una gráfica del tiempo frente a k para las dos soluciones. Antes de implementar Held-Karp, justificar que el problema tiene subestructura óptima y plantear la recurrencia. Held-Karp debe devolver el recorrido (el orden de visita), no solo su tiempo. Estimar cuánta memoria usa según el tamaño de su tabla. Verificar que ambos métodos den el mismo tiempo total óptimo en los valores de k que la fuerza bruta alcance a resolver.
2. Heurística. Implementar (implementación propia) una heurística voraz para construir el recorrido, saliendo del lugar de partida y regresando a él, elegida por el grupo, y justificar la elección.
3. Calidad de la heurística. Para k = 5, 8, 10 y 12, con al menos 5 instancias por cada k, reportar cuánto se aleja la heurística elegida del óptimo de Held-Karp (en porcentaje, promedio y máximo).

4. Itinerario de un día. Elegir un lugar de partida del catálogo (por defecto, Armenia) y entre 6 y 8 atractivos, y ordenarlos con Held-Karp, saliendo de ese lugar y regresando a él. Comprobar si la duración de las visitas más los desplazamientos cabe en una jornada de 11 h; si no cabe, quitar atractivos justificando cuáles, recalcular el recorrido y reportar su duración final.

5. Voraz frente a óptimo. Mostrar con un ejemplo por qué la heurística elegida no siempre encuentra el mejor recorrido. Comparar con Dijkstra, que también es voraz: explicar por qué allí la elección voraz sí garantiza el óptimo y aquí no.

Requerimiento 5. Aplicación

La aplicación es demostrativa: se evalúa que muestre los resultados del proyecto, no su diseño visual. Puede usar la envolvente ya calculada; las rutas entre origen y destino y el itinerario se calculan en el momento, con Dijkstra con cola de prioridad, con Held-Karp y con la heurística voraz.

1. La aplicación (web o de escritorio) debe permitir:
   - elegir origen y destino y ver la ruta en un mapa, con su tiempo de viaje;
   - ver la envolvente convexa sobre el mapa del departamento;
   - elegir el lugar de partida del itinerario (por defecto, Armenia) y entre 6 y 8 atractivos, y ver el recorrido del día obtenido con Held-Karp y con la heurística voraz, con el tiempo total de cada uno y la diferencia entre ellos;
   - ver las gráficas y tablas de resultados de los requerimientos 1 a 4 (pueden ser las mismas imágenes del documento técnico).

- Debe poder ejecutarse localmente siguiendo el README. Publicarla en una URL o en un contenedor Docker es opcional.

6. Protocolo experimental

Todas las mediciones siguen estas reglas:

1. Las mediciones de un mismo requerimiento se hacen en la misma máquina. Se reportan procesador, memoria RAM, sistema operativo y versión del lenguaje.

2. Solo se mide el algoritmo: leer archivos, generar datos e imprimir resultados quedan por fuera.

3. Cada medición se repite 5 veces y se reporta la mediana. Las ejecuciones de más de un minuto se miden una sola vez.

4. Los tiempos se guardan en una misma unidad (milisegundos o nanosegundos).

5. Los datos aleatorios se generan con una semilla fija, que se documenta.

6. Antes de medir, cada algoritmo se verifica: contra otra implementación propia, contra un caso pequeño resuelto a mano o contra una librería. En el documento técnico se explica cómo se verificó cada uno. Un algoritmo rápido pero incorrecto no cuenta como implementado.

7. Límite de tiempo: si una ejecución pasa de 5 minutos, se detiene y se reporta como no completada.

8. Tablas y gráficas: los resultados se guardan en archivos CSV o JSON, y las tablas y gráficas que pide cada requerimiento se generan a partir de esos archivos. Cada una indica la unidad de tiempo y va acompañada de un párrafo que explique si lo medido coincide con lo esperado y, si no, por qué. El anexo muestra el formato de cada tabla, archivo y gráfica.

7. Entregables

Entregable	Contenido
Repositorio en GitHub	Código, datos procesados, resultados de los experimentos y un README con instrucciones para reproducirlos
Documento técnico	Ver estructura abajo
Aplicación	Ejecutable localmente siguiendo el README
Sustentación	Presentación del grupo y preguntas individuales

Estructura del documento técnico:

1. Arquitectura de la solución: un diagrama de componentes y cómo pasan los datos de un requerimiento a otro.
2. Por cada requerimiento: el problema, la complejidad de cada algoritmo en tiempo y en memoria con la fuente (y su recurrencia, cuando corresponda), los argumentos de correctitud que se piden, los resultados en tablas y gráficas, y su discusión.
3. Conclusiones.
4. Anexo de uso de inteligencia artificial.

8. Evaluación

La nota es grupal y se ajusta para cada integrante según su sustentación.

Componente	Peso
R1. Modelo de la red vial	15 %
R2. Caminos mínimos	20 %
R3. Envolvente convexa	20 %
R4. Ruta óptima de recorrido	25 %
R5. Aplicación	5 %
Documento técnico (organización, claridad y rigor)	15 %

En R1 a R4, la nota de cada requerimiento se reparte así: 40 % implementación correcta y verificada, 30 % análisis teórico (complejidad y argumentos de correctitud) y 30 % análisis experimental. En R5 se evalúa que la aplicación muestre los resultados y funcione siguiendo el README.

Nota individual. Cada integrante responde preguntas sobre cualquier parte del proyecto, no solo la que desarrolló. Su nota es la nota del proyecto multiplicada por un factor de sustentación entre 0 y 1.